

The Production AI Engineer Roadmap — 1 Month, Project-First

Before the plan, three things I need to say as someone who'd be reviewing your application.

The market is real and the window is specific. Across every June 2026 posting I just read, the same stack repeats: RAG + vector DBs, agent orchestration (LangGraph/CrewAI), tool calling + MCP, evaluation pipelines, guardrails, cost/latency optimization, and production observability. Agent orchestration, MCP integration, eval design, prompt engineering, vector DB/RAG, cost optimization, safety/guardrails, computer-use deployment, production observability, and frontier-model fluency are the 2026 hiring checklist. Your instinct is correct — and the corollary matters: candidates who list only LangChain, Pinecone, and ChatGPT API without these newer capabilities are likely in a 2024 mindset. The thing that gets you neglected isn't a missing skill, it's a portfolio that looks like everyone else's.

Hiring is link-first now. Recruiters don't ask for resumes first anymore. They ask for links. GitHub repositories, live demos, and deployed apps matter more than titles or years of experience. And the failure mode is specific: most rejections happen not due to weak skills, but because resumes fail to show impact fast. This is why your "unique, not-on-GitHub, live demo" framing is exactly right.

One month is aggressive but workable *because* of your background. Strong DSA + Python means the AI-specific layer is genuinely the only gap. The gap is specific: LLM APIs, RAG, agents, and evaluation. But be honest with yourself about hours — this plan assumes ~6 hrs/day, 6 days/week. If you have less, stretch it to 6–7 weeks rather than cutting scope.

A note on philosophy: you asked to learn mostly through one production project. I'm giving you **two**, but they're sequenced — Project 1 (weeks 1–2) teaches RAG + production rigor; Project 2 (weeks 3–4) teaches agents + MCP + multi-agent and *absorbs* Project 1 as a tool. So it's really one growing system. That's deliberate — it tells a story a reviewer can follow, and the second project demonstrates you can build *on* your own infrastructure, which is what senior engineers actually do.

Part 1: The Depth Map — What to Study and How Deeply

This is the part most roadmaps get wrong. Here's the honest calibration for a *production* AI engineer, not a researcher.

Transformers & research papers

Topic	Depth needed	Why
Attention Is All You Need	Read once, conceptually.	You need the mental model to reason about context limits, latency, and cost — not to reimplement it. ~3 hours, one time.
	Understand: attention = weighted lookup over tokens, why context windows are finite, why position matters. Do NOT derive the math.	
How transformers work (inference)	Moderate. KV-cache, why prefill vs decode differ, why long context is expensive, tokenization.	This directly explains caching strategies, latency optimization, and cost — which <i>are</i> on the job.

Topic	Depth needed	Why
Fine-tuning / LoRA / QLoRA	Conceptual only. Know <i>when</i> you'd fine-tune vs RAG vs prompt (almost always you won't fine-tune at a startup early on). Know it exists, what it costs, what it buys.	Interviewers ask "would you fine-tune here?" The correct answer is usually "no, here's why RAG/prompting is cheaper and faster to iterate." Knowing that <i>is</i> the skill.
RLHF / DPO	Awareness only. One paragraph: it's how base models become assistants.	You will never touch this at an application startup. Being able to say one sentence about it is plenty.
Quantization (int8/int4, GPTQ/AWQ)	Conceptual + practical <i>if</i> self-hosting. Know it trades quality for speed/cost.	Relevant only when you serve open models via vLLM. Most startups use APIs; know it for the cases that don't.
Embeddings (how they're trained)	Conceptual. You need to <i>use</i> and <i>evaluate</i> them, not train them.	Embedding <i>selection</i> and <i>evaluation</i> are on the job; embedding <i>training</i> is not.

The rule: study theory only to the depth that lets you make a *production decision*. If a concept doesn't change a decision you'd make about an architecture, cost, or failure mode, you don't need its math.

Essential / Optional / Unnecessary

ESSENTIAL (the spine of your employability):

- LLM API fluency — Anthropic + OpenAI, streaming, tool calling, structured outputs. LLM API mastery: Anthropic, OpenAI, Bedrock — knowing one is not enough.
- Prompt & context engineering — treated as a software contract, not a vibe. System prompts are software contracts now.
- RAG end-to-end: chunking, embeddings, hybrid retrieval, re-ranking, metadata filtering
- Vector DBs: **pgvector** + **Qdrant** (pick these two — see tooling section)
- Agent orchestration: **LangGraph** (the one to actually learn deeply)
- Tool/function calling, JSON schema design, tool contract design
- **MCP** — this is the highest-leverage *newer* skill; it's all over the postings
- Evaluation: LLM-as-judge, RAG metrics, regression testing (**Ragas** + **LangSmith**)
- Guardrails: prompt injection defense, PII handling, input/output validation, the "**Excessive Agency**" failure mode
- Reliability: retries, exponential backoff, timeouts, circuit breakers, graceful degradation
- Core SWE: **Python async**, **FastAPI**, **Postgres**, **Redis**, **Docker**, **GitHub Actions CI/CD**
- Observability: tracing agent decisions, structured logging, metrics

- Cost & latency optimization (caching, model routing, token budgets)
- Product thinking: communicating uncertainty, graceful failure, feedback loops

OPTIONAL (learn if time allows, or learn on the job):

- **CrewAI** — learn the *concept* of role-based multi-agent, but build in LangGraph. CrewAI is easy to pick up later; one afternoon reading docs. LangGraph and CrewAI replaced ad-hoc loops — but LangGraph is the deeper, more transferable skill.
- **vLLM / self-hosting / quantization** — only if you target startups serving open models. Most use APIs. Know the concepts; build only if you have spare time in week 4.
- **Kubernetes** — explicitly optional. Docker + a PaaS (Railway/Render/Fly) is enough to ship. K8s is a "nice to mention you understand it conceptually" item. Don't spend project time here.
- **Pinecone** — it's a managed Qdrant-alike. If you know Qdrant + pgvector, you can speak to Pinecone in an interview without building on it. Skip building; read the docs.
- **Twingate** — niche (secure network access to internal resources). One paragraph of awareness for enterprise-data questions. Not a build item.
- Knowledge graphs / GraphRAG — emerging, mentioned in a few EU postings. Awareness only for now.

UNNECESSARY (actively skip for your goals):

- Deriving transformer math, training models from scratch, RL research, DeepSpeed/Ray distributed training, CUDA. These are the *researcher* resume (you saw one in the search — Ray/DeepSpeed/CUDA is the FAANG-research track, **not** the startup-application track). Ignore that path entirely.
- Classic ML (XGBoost, feature stores, drift detection on tabular models) — different role (ML Engineer). Don't dilute your story.

HLD vs LLD — how much, relative to backend engineers

Here's the honest comparison you asked for:

High-Level Design: you need ~80% of what a backend engineer needs, plus an AI-specific layer they don't have. The shared part (services, queues, caching, API gateways, event-driven patterns, reliability) is identical — and you should learn it because AI systems *are* distributed systems. The AI-specific HLD layer that's uniquely yours: retrieval architecture, where evaluation sits in the pipeline, how you handle non-deterministic outputs at system boundaries, cost/latency tradeoffs of model calls, and human-in-the-loop checkpoints. **This is where you out-design a generalist backend engineer** — they don't think about eval gates or excessive-agency containment.

Low-Level Design: you need less than a backend engineer, and only a focused slice. You don't need the full Gang-of-Four catalog. The LLD that earns its place for AI work:

- **SOLID** — specifically so your tool/agent code stays testable

- A handful of patterns: **Strategy** (swappable models/retrievers), **Adapter** (wrapping external APIs/tools), **Factory** (provider selection), **Chain of Responsibility** (guardrail pipelines), **Circuit Breaker**, **repository**
- **Tool abstraction design** and **interface contracts** — this is the LLD that's genuinely AI-specific and genuinely tested
- Clean code organization so a reviewer reading your repo sees maturity

Don't grind LeetCode-style LLD or memorize 23 patterns. Learn the six above *by applying them in the projects*. The interview signal isn't "can you recite patterns" — it's companies aren't hiring people who can "use an LLM API," they're hiring people who can reason about constraints, trade offs, and failure modes the way real engineers do.

The tooling stance (since you need clear answers, not a menu)

The single most useful clarification for 2026: **LangChain and LangGraph are not competitors**. Since October 22, 2025, LangChain's own `create_agent` function runs on LangGraph's execution engine under the hood. Practical guidance: use `create_react_agent()` for prebuilt patterns or LangGraph's StateGraph for custom orchestration, and note that AgentExecutor is in maintenance mode until December 2026 — so never learn the old pattern.

My recommendation for *learning*: **learn the raw SDK first, then LangGraph**. Build your first tool-calling loop with the bare Anthropic/OpenAI SDK so you understand what's actually happening, *then* adopt LangGraph for orchestration. This matters because — and this is a real interview differentiator — most teams that argue about LangGraph vs LangChain are optimizing the wrong layer. The bottleneck is almost never the orchestration framework. It's TTFT (time to first token), throughput, and cost per token from the inference layer. An engineer who knows the framework is a convenience, not magic, reasons better about systems.

Your committed stack: Python/async, FastAPI, Postgres + pgvector, Qdrant, Redis, LangGraph (+ a little CrewAI), MCP, Ragas, LangSmith, Docker, GitHub Actions. **Optional/awareness:** vLLM, Pinecone, Kubernetes, Twingate, CrewAI-as-primary.

Part 2: The Week-by-Week Order (Exact Sequence)

The ordering principle: **never learn a concept you won't use within 48 hours**. Each block feeds the project you're building that week.

Week 0 — Setup weekend (2 days, before the clock starts)

1. Get API keys (Anthropic + OpenAI), set up a clean repo with uv or Poetry, pre-commit hooks, ruff.
2. **Python async crash course** — `async/await`, `asyncio.gather`, why it matters for LLM I/O. (You know Python; this is the one language gap that's load-bearing — `async` is what separates toy throughput from real.) Without `async`, throughput dies.
3. **FastAPI fundamentals** — routes, Pydantic models, dependency injection, streaming responses, background tasks.

4. First "hello LLM" script: a streaming chat call with the raw SDK, then the same with tool calling and structured output (Pydantic). Feel the primitives before any framework.

Week 1 — RAG foundations + the spine of Project 1

Concepts, in order:

1. **Embeddings** — what they are, how to call an embedding model, cosine similarity by hand once.
2. **Embedding model selection & evaluation** — how to compare models (MTEB awareness, but mostly: test on *your* data).
3. **Chunking strategies** — fixed, recursive, semantic; why chunk size is a tradeoff; metadata attachment.
4. **Vector search** — set up **pgvector** first (you get Postgres + vectors in one DB, the pragmatic startup default), then **Qdrant** (to learn a dedicated vector DB and metadata filtering properly).
5. **Hybrid retrieval** — keyword (BM25) + semantic, why hybrid beats either alone. Context Relevance: Was the retrieved data actually useful?
6. **Re-ranking** — cross-encoder rerankers, when they're worth the latency.
7. **Metadata filtering** — pre/post-filtering, access-control-aware retrieval (sets up your security story).
8. Wire all of it into a FastAPI service. This is Project 1's core.

Week 2 — Production rigor on top of Project 1

Concepts, in order:

1. **Evaluation** — this is the skill that separates you. **Ragas** for the RAG triad: Faithfulness: Does the answer contradict the retrieved source? Answer Relevance: Does it actually answer the user's question? Context Relevance: Was the retrieved data actually useful?
2. **LLM-as-a-Judge** — and critically, its failure mode: How do you prevent "Identity Bias" where an LLM prefers its own style? (Knowing this question's answer is a real interview edge.)
3. **Regression testing & benchmark creation** — build a golden dataset, gate CI on eval scores.
4. **Observability** — **LangSmith** tracing every retrieval + generation step. Using LangSmith or Arize Phoenix to "trace" every step.
5. **Defensive engineering** — retries, exponential backoff, timeouts, circuit breakers, graceful degradation when retrieval or the LLM fails.
6. **Guardrails** — prompt injection defense, input/output validation, **PII detection & redaction** before text hits the LLM or logs.
7. **Caching & cost** — semantic caching with Redis, prompt caching, token budgeting, model routing (cheap model for easy queries).
8. **Docker + GitHub Actions** — containerize, CI runs tests + evals, deploy to a PaaS with a **live demo URL**.

9. Write the **case-study README** + record a **2–3 min Loom**. Aim for live demos that load in ~3 seconds, a 2-3 minute Loom walkthrough, and a case-study README explaining problem, architecture, and tradeoffs.

Week 3 — Agents, tools, MCP + start Project 2

Concepts, in order:

1. **Tool/function calling deeply** — JSON schema design, **tool contract design**, validation, error surfaces back to the model.
2. **The agent loop from scratch** — build a ReAct-style loop with the raw SDK (~50 lines) so you understand what LangGraph abstracts.
3. **LangGraph** — StateGraph, nodes, edges, conditional routing, loops, durable state. If your agent needs loops, retries, or multi-turn decisions, LangGraph is recommended.
4. **Durable state** — in-memory agents are toys: production agents require durable state (Postgres/Redis) with Pydantic schemas; LangGraph checkpoints.
5. **Memory systems** — short-term (conversation) vs long-term (retrieved); summarization for long-context management.
6. **MCP (Model Context Protocol)** — build an MCP server exposing your Project 1 RAG system as a tool. This is the integration pattern showing up across postings: experience implementing and extending the Model Context Protocol (MCP) to provide LLMs with secure, standardized access to local and remote data sources.
7. **Human-in-the-loop** — LangGraph interrupt() for approval checkpoints on consequential actions.

Week 4 — Multi-agent, hardening, ship Project 2

Concepts, in order:

1. **Multi-agent collaboration** — supervisor/router pattern, when multi-agent helps vs hurts (usually a single well-tooled agent beats a crowd). Read CrewAI's role-based model here for breadth.
2. **The Excessive Agency failure mode** — and mitigations. This is a senior signal: propose mitigations (least-privilege tool design, human-in-the-loop checkpoints, kill switches).
3. **Security hardening** — authn/authz, secrets management, **safe tool execution** (sandboxing, permission boundaries), output filtering.
4. **Reliability at the system level** — idempotency, dead-letter queues, fault isolation, external-API failure handling.
5. **Agent evaluation** — task success rate, trajectory evaluation, tracing *why* an agent chose a tool. "Where did the agent decide to call the wrong tool?"
6. **Cost/latency for agents** — token accounting per run, parallel tool calls, streaming.
7. **Product thinking pass** — communicating uncertainty in the UI, graceful degradation, feedback capture.

- Ship Project 2 with live demo, README, Loom. Then write **one LinkedIn build post per project** — every AI portfolio project needs four things: a production-quality README, a live demo URL, a LinkedIn post about the build process, and a "Technical Decisions" section that demonstrates engineering judgment.

Part 3: The Two Projects (Designed to Be *Not* Generic)

Here's the core problem with most portfolios: most AI portfolios are toy demos, and GitHub hosts over 4.3 million AI repositories, and the 2025 Octoverse report notes a 178% year-over-year increase in LLM-focused projects. A "chat with your PDF" RAG app is invisible. The way you stand out is **a specific, opinionated domain + production rigor most demos skip.**

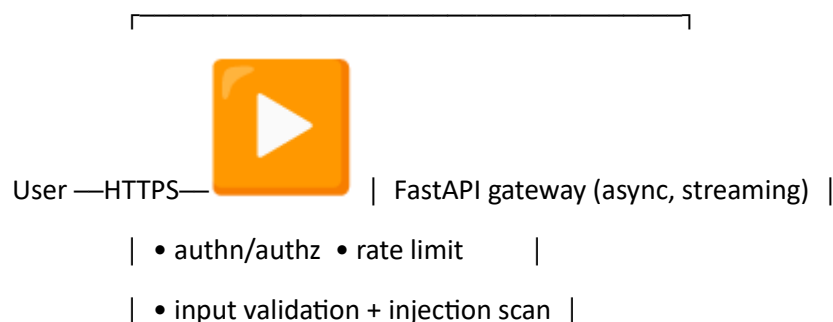
Both projects below avoid the generic trap by: (a) operating on a domain with real structure and stakes, (b) shipping the things tutorials omit — eval suites, guardrails, observability, cost controls, and (c) being honestly framed around a problem, not a tool. The point a reviewer sees: make AI transparent (show sources, confidence, and cost controls) so your project signals engineering judgment rather than a gimmick.

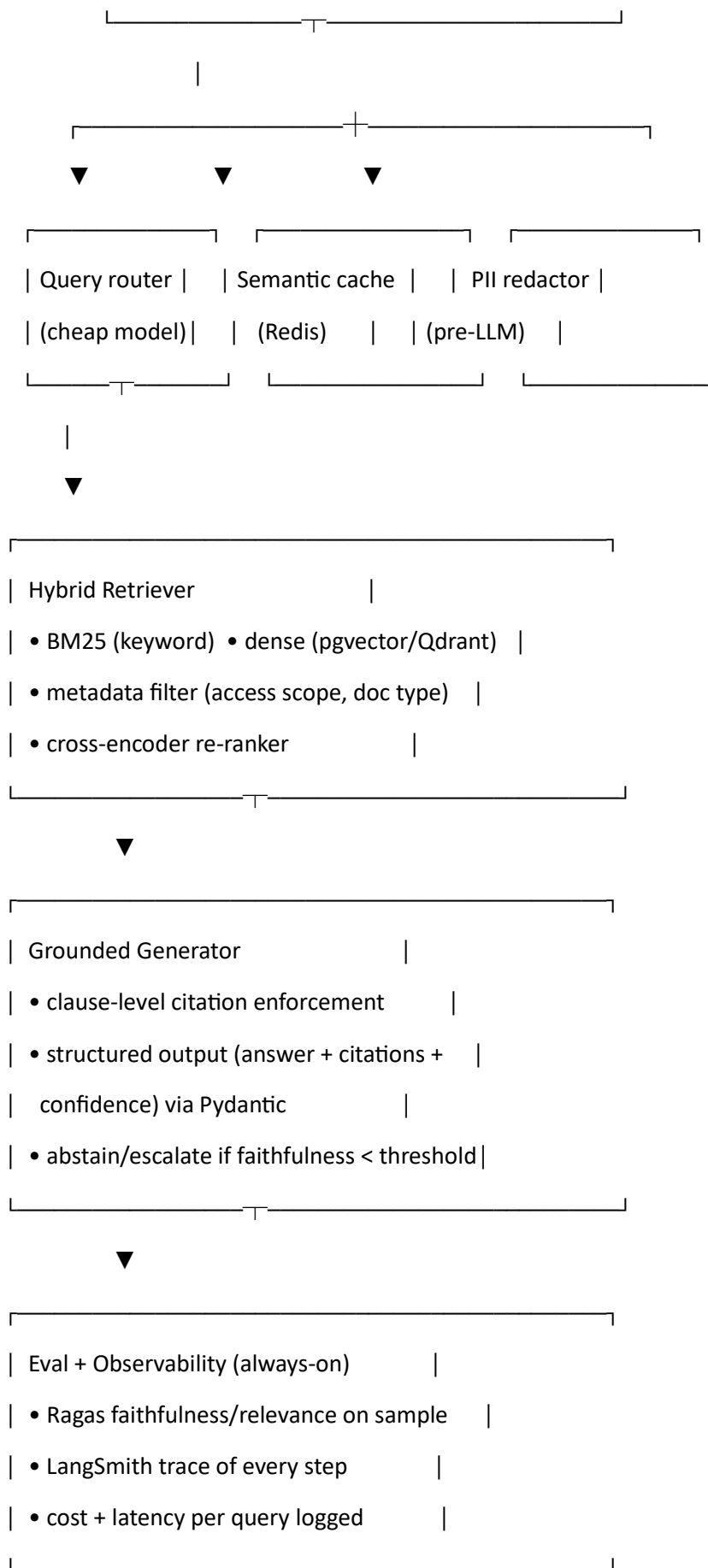
PROJECT 1: "Clause" — A Contract & Policy Intelligence System with Citations and a Confidence Budget

Problem statement. Teams (legal ops, procurement, compliance) need to ask natural-language questions across a corpus of contracts, policies, and regulations and get answers that are *grounded, cited to the exact clause, and honest about uncertainty* — because a confidently wrong answer about an indemnification clause is worse than no answer. Generic RAG demos return a paragraph and a vibe. This system returns an answer, the exact source clauses, a faithfulness score, and refuses (or escalates) when grounding is weak.

Why this isn't generic / why a startup HR won't neglect it. It's not "chat with a PDF." It's a domain where *being wrong has consequences*, so it forces you to build the production rigor that's actually the job: clause-level citations, an eval harness that measures faithfulness, PII/sensitive-data handling, access-control-aware retrieval, and explicit uncertainty communication. RAG is also the single most hireable pattern: the most effective project for immediate hiring is a Retrieval Augmented Generation System because it solves common business problems by grounding language models in specific enterprise data for accuracy. You're doing the hireable pattern *with the rigor most candidates skip.*

Architecture overview.





CI/CD: GitHub Actions → tests + eval-gate → Docker → PaaS (live demo)

Data: ingestion pipeline (parse → clause-aware chunk → embed → index)

Component breakdown.

- **Ingestion pipeline** — document parsing, *clause-aware* chunking (not blind 512-token splits — split on legal/policy structure), embedding with a selected + evaluated model, dual-indexing into pgvector and Qdrant so you can demo both and discuss tradeoffs.
- **Hybrid retriever** — BM25 + dense + RRF fusion, metadata pre-filtering for access scope, cross-encoder reranking with a latency budget.
- **Grounded generator** — structured output (answer, citations[], confidence, abstained: bool); a self-check pass that verifies each claim maps to a retrieved clause.
- **Confidence budget** — if faithfulness or retrieval score is below threshold, the system abstains and says so rather than guessing. This *is* your "communicating uncertainty" story made concrete.
- **Eval harness** — a golden dataset of Q→expected-grounding pairs; Ragas metrics; CI fails if faithfulness regresses.
- **Guardrail layer** — injection scanning on input, PII redaction before LLM + before logs, output filtering.
- **Cost/latency layer** — semantic cache, query router (route trivial queries to a cheap model), per-query token + cost accounting surfaced in the UI.

Technologies. Python/async, FastAPI, pgvector + Qdrant, Redis, an embedding model + cross-encoder reranker, Anthropic/OpenAI SDK, Ragas, LangSmith, Docker, GitHub Actions, a PaaS for the live demo. (LangGraph *not* required here — keep Project 1 framework-light to prove you understand the primitives; that's a deliberate talking point.)

Advanced features beyond tutorial level. Clause-level citation enforcement with self-verification; abstention/escalation on low confidence; hybrid + rerank (most demos do pure-vector); access-control-aware retrieval; semantic caching; model routing for cost; a real eval suite gating CI.

Reliability considerations. Timeouts + exponential-backoff retries on every external call; circuit breaker around the LLM provider with a degraded fallback (return retrieved clauses *without* generation if the model is down — still useful); graceful "I can't answer confidently" path.

Security considerations. Input validation + prompt-injection scanning; PII detection and redaction pre-LLM and pre-log; metadata-enforced permission boundaries so users only retrieve documents in their scope; secrets via environment/secret manager, never in code. This directly answers understanding of data privacy and handling of sensitive data (PII/PHI).

Evaluation methodology. Golden dataset of realistic queries with known correct grounding; Ragas faithfulness/answer-relevance/context-relevance; LLM-as-judge for answer quality *with identity-bias mitigation* (use a different model family as judge than as generator); regression gate in CI; a small held-out adversarial set (injection attempts, out-of-corpus questions to test abstention).

Observability strategy. LangSmith trace of retrieve→rerank→generate→verify for every request; structured logs with a request ID; per-query metrics: retrieval score, faithfulness, latency breakdown (retrieval vs generation), tokens, cost. A simple /metrics dashboard or a Grafana-style view is a strong bonus.

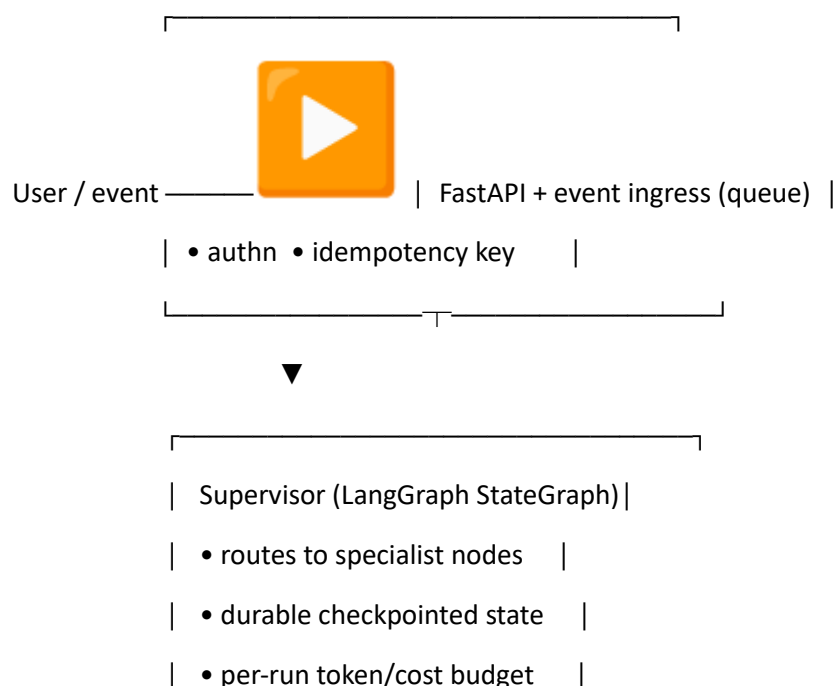
Deployment architecture. Dockerized service + Postgres/pgvector + Redis via Docker Compose locally; GitHub Actions pipeline (lint → test → **eval-gate** → build image → deploy); hosted on Railway/Render/Fly with a public demo URL and a seeded demo corpus (use public contracts/policies so anyone can try it in 30 seconds). The eval-gate-in-CI is the detail that makes a hiring manager stop scrolling.

PROJECT 2: "Operator" — A Multi-Step Agent that Safely Acts on Your Internal Systems (with MCP, HITL, and a Kill Switch)

Problem statement. Startups want agents that don't just *answer* but *do* — triage an incoming request, look things up across internal data, draft an action (issue a refund, file a ticket, update a record, send a reply), and execute it — *safely*. The hard part isn't the LLM; it's making an acting agent that has least-privilege access, asks a human before consequential/irreversible actions, recovers from failures mid-run, and can be audited and stopped. Most "agent" demos are an LLM in a while loop with no guardrails. This one is built like something a company could actually turn on.

Why this isn't generic / why a startup HR won't neglect it. This is precisely the architecture the market says it's betting on: a control panel that shows agents talking to each other, calling tools, and tracking costs looks like someone who understands where AI products are actually heading. And it's built around the *failure mode senior interviews probe* — LLM06 Excessive Agency — the agent-specific failure mode where a system takes irreversible actions beyond its intended scope — with the exact mitigations they want to hear: least-privilege tools, HITL checkpoints, kill switches. Reusing Project 1 as an MCP-exposed retrieval tool shows you build *on* your own systems.

Architecture overview.



- **Action agent + HITL gate** — drafts consequential actions and *pauses* via LangGraph `interrupt()` for human approval before anything irreversible; reversible actions can auto-execute.
- **Safe tool executor** — every tool has least-privilege scope, an allowlist, input validation; failures route to a **dead-letter queue** for inspection/retry.
- **Kill switch** — a global stop that halts in-flight runs (the "kill switch" interviewers want to hear about).
- **Eval harness** — trajectory evaluation (did it take a sensible path?) + task-success rate on a scenario suite, including adversarial scenarios (does it try to skip the HITL gate? does it stay in scope?).

Technologies. Python/async, FastAPI, **LangGraph** (StateGraph, checkpointers, interrupt), **MCP** (server + client), Postgres (durable state) + Redis (queue/cache), a message queue or task runner for events + dead-letter, LangSmith, Docker, GitHub Actions, PaaS demo. CrewAI optionally referenced in the README as the alternative you evaluated and why you chose LangGraph (breadth signal).

Advanced features beyond tutorial level. MCP-based tool integration (not hardcoded function calls); durable checkpointed state surviving restarts; HITL approval gates; per-run cost budget + kill switch; dead-letter queue with replay; least-privilege tool scoping; trajectory-level evaluation. A strong orchestrator shows you can coordinate multiple AI roles, keep shared state consistent, and recover gracefully when things fail.

Reliability considerations. Idempotency keys so a retried event doesn't double-execute an action (critical for anything with side effects); retries with backoff + circuit breakers per tool; dead-letter queue + replay; durable state for resume-after-failure; fault isolation so one failing tool doesn't crash the run.

Security considerations. Least-privilege scopes per tool; HITL approval before irreversible actions; sandboxed/allowlisted tool execution; authn/authz on the gateway; secrets management; output filtering; audit log of every action taken. This is your direct, demonstrable answer to Excessive Agency.

Evaluation methodology. Scenario suite with known-correct trajectories; task-success rate; trajectory grading (LLM-as-judge with a different model family); **adversarial scenarios** — attempts to bypass HITL, act out of scope, or exceed budget — and assert the system blocks them. Tracing *why* a tool was chosen, per "Where did the agent decide to call the wrong tool?"

Observability strategy. Full LangSmith trace of every node, decision, and tool call with a run ID; per-run token + cost; a live **control-panel UI** showing the agent's steps, current state, pending approvals, and cost in real time. That dashboard is the "stop scrolling" artifact — it's the visible proof of the architecture.

Deployment architecture. Docker Compose (app + Postgres + Redis + the MCP server) locally; GitHub Actions pipeline with an **agent-eval gate**; deployed to a PaaS with a public demo where a visitor can trigger a safe scenario, watch the trace, and approve an action through the HITL gate. A guest/demo mode so a reviewer sees value in under 30 seconds.

I'm keeping this tight — a giant list is noise. These are the high-signal ones per area.

Foundational / LLM APIs & prompting

- Anthropic's docs + prompt engineering guide, and OpenAI's API docs — primary sources, read directly.
- Anthropic's "Building effective agents" guide — short, opinionated, exactly your level.
- DeepLearning.AI short courses (free, ~1–2 hrs each): "ChatGPT Prompt Engineering for Developers," "Functions/Tools," "Building Systems with the ChatGPT API."

RAG & vector search

- DeepLearning.AI: "Building and Evaluating Advanced RAG" (this maps almost 1:1 to Project 1).
- Qdrant docs + pgvector README — primary sources; build from these.
- Pinecone's learning center has genuinely good *conceptual* RAG/embedding articles even though you won't build on Pinecone.

Agents & LangGraph

- LangGraph official docs + their "agents from scratch" / academy material — the authoritative source given how fast it moves.
- Anthropic's MCP documentation + the MCP spec site — primary source for Project 2's showpiece.

Evaluation

- Ragas documentation (primary).
- LangSmith docs + their evaluation cookbook.

System design / SWE maturity

- For HLD: "Designing Data-Intensive Applications" (Kleppmann) — don't read cover to cover; read the chapters on replication, partitioning, and reliability. This is your shared-with-backend foundation.
- For the AI-specific design layer: Chip Huyen's "AI Engineering" (2025) book — the single most aligned book to your exact goal; read it in full.
- FastAPI official docs + "FastAPI" tutorial — primary source, excellent.

Blogs / ongoing

- Anthropic and OpenAI engineering blogs; Chip Huyen's blog; Eugene Yan's blog (ML/LLM systems in production); the LangChain blog for framework changes.

YouTube

- For concepts at your depth: 3Blue1Brown's neural network/transformer series *once* (visual intuition, zero math grind). For practical: official LangChain channel and freeCodeCamp's longer LLM/RAG build-alongs.

GitHub repos to study (read, don't copy)

- The official LangGraph examples repo and the MCP servers reference repo — to see idiomatic patterns.
- Ragas examples — to see eval harnesses structured properly.

A caution on AI-assisted building, because you'll use Cursor/Claude Code and should: use AI as a power tool for scaffolding, tests, and brainstorming, but never copy outputs verbatim; you must be able to explain and modify what the model produced. In interviews you *will* be asked to defend a design decision in your own repo. Build so you can.

Part 5: The Honest Caveats

A few things I'd be doing you a disservice to leave out:

One month gets you to "interview-ready with two strong projects," not to "senior." That's the correct goal and it's achievable. Depth in agents, eval, and production failure modes keeps compounding for months after — keep going past day 30, especially on the vLLM/self-hosting and deeper system-design material you deferred.

The projects matter more than the checklist. If you're short on time, cut breadth, not project depth. Two deployed, well-documented, *defensible* systems with live demos beat ten half-finished ones every time. Quality and deployment matter more than a high volume of unfinished experiments.

Pace yourself on the second project. Project 2 is genuinely harder (durable state, MCP, HITL, dead-letter queues). If week 4 runs over, it's fine to ship a tighter version (single specialist agent + HITL + MCP + the kill switch + eval) and mark the multi-agent supervisor as "v2 in progress." A reviewer respects honest scoping.

My read on the market is from sources dated through this week, and the tooling (LangGraph/MCP especially) moves monthly — always check the official docs for the current API before building, since AgentExecutor-style tutorials from 2024 are now actively the *wrong* thing to learn.